

파이썬의 숨겨진 함정들

Python WTF Moments

숭실대학교 소프트웨어학부 21 김현수

간단한 문제부터 시작해보죠

4장의 카드가 있고, 각 카드는 한 면에 숫자, 다른 면에 색깔이 있습니다.

규칙: 카드에 짝수가 적혀져있다면, 뒷면은 파란색이다.

이 규칙이 참인지 거짓인지 확인하려면 어떤 카드들을 뒤집어야 할까요?

3 8 파랑 빨강

정답은?

정답: 8, 빨강

- 3 : 홀수라서 규칙과 무관
- 8 : 짝수 → 뒷면이 파란색인지 확인 필요
- 파랑 : 앞면이 무엇이든 규칙 위반 아님
- 빨강 : 앞면이 짝수라면 규칙 위반!

Wason Selection Task

- 1966년 Peter Wason이 개발한 논리 추론 실험
- 조건문 검증 능력 테스트 ("If P then Q" 형태)
- 반증 가능성을 찾는 능력 측정
- 인지심리학에서 가장 많이 연구된 과제

충격적인 사실: 대학생의 90% 이상이 틀림

같은 논리, 다른 맥락

"술집에서 음주자는 21세 이상이어야 한다"

맥주	콜라	25세	16세
----	----	-----	-----

이번엔 쉽게 답을 찾으시죠?

- 맥주: 마시는 사람이 21세 이상인가?
- 16세: 술을 마시고 있지는 않나?

사회적 맥락에서는 더 잘 해결합니다!

System 1 vs System 2

Daniel Kahneman의 연구에 따르면:

- System 1: 직관적, 휴리스틱 기반 (빠름, 자동)
- System 2: 논리적-분석적 (느림, 의식적, 에너지 소모)

우리는 System 1을 선호하지만, 코딩에서는 System 2가 필요!

 **해결책:** System 2를 활성화하는 코딩 습관을 들여 System 1 영역으로 이전

Python WTF Moments

파이썬의 함정들을 통해 논리적 사고를 연습해봅시다!

Walrus Operator (바다코끼리 연산자)

Python 3.8+의 새로운 기능

```
>>> a = "wtf_walrus"
>>> a
'wtf_walrus'

>>> a := "wtf_walrus" # 에러!
SyntaxError: invalid syntax

>>> (a := "wtf_walrus") # 괄호 필요!
'wtf_walrus'
>>> a
'wtf_walrus'
```



Walrus Operator - 튜플의 함정

```
>>> a = 6, 9
```

```
>>> a  
(6, 9)
```

```
>>> (a := 6, 9)  
(6, 9)
```

```
>>> a  
6 # a는 6만 할당됨!
```

```
>>> (a, b := 16, 19) # 이상한 3-tuple  
(6, 16, 19)
```

```
>>> a # a는 여전히 6  
6
```

```
>>> b # b만 16으로 할당  
16
```

Walrus Operator - 실용적 사용

```
def some_func():  
    # 비싼 연산이라고 가정  
    return 5  
  
# 나쁜 예: 두 번 계산  
if some_func():  
    print(some_func())  
  
# 기존 방법  
a = some_func()  
if a:  
    print(a)  
  
# Walrus 연산자 사용  
if a := some_func():  
    print(a) # 한 번만 계산!
```

할당 연산자 vs Walrus 연산자

구분	할당 연산자 (<code>=</code>)	Walrus 연산자 (<code>:=</code>)
타입	문(Statement)	표현식(Expression)
반환값	없음	할당한 값 반환
중첩 사용	불가능	가능
언패킹	지원	미지원

틱택토 X의 승리

다음 코드의 결과를 예상해보세요:

```
# 행 초기화
row = [""] * 3 # ['', '', '']
# 보드 만들기
board = [row] * 3

>>> board
[['', '', ''], ['', '', ''], ['', '', '']]
>>> board[0][0] = "X"
>>> board
```

결과는? 🤔

틱택토 - 예상과 다른 결과

```
>>> board[0][0] = "X"  
>>> board  
[['X', '', ''], ['X', '', ''], ['X', '', '']]
```

모든 행의 첫 번째 칸에 'X'가! 🤖

원인: `[row] * 3` 은 같은 리스트 객체를 3번 참조

→ 얕은 복사(Shallow Copy) 문제

올바른 해결책

```
# 방법 1: 리스트 컴프리헨션
board = [["_"] * 3 for _ in range(3)]

# 방법 2: 깊은 복사
import copy
board = copy.deepcopy(["_"] * 3) * 3)
```

기억하세요: * 연산자는 객체를 복사하지 않고 참조만 복사합니다!

가변 기본 인수의 함정

파이썬에서 가장 유명한 함정:

```
def add_item(item, lst=[]):  
    lst.append(item)  
    return lst  
  
>>> print(add_item('사과'))  
['사과']  
>>> print(add_item('바나나'))  
['사과', '바나나'] # 어? 사과가 왜 있지?  
>>> print(add_item('오렌지'))  
['사과', '바나나', '오렌지'] # 계속 누적됨!
```

왜 이런 일이?

기본 인수는 함수 정의 시점에 한 번만 생성되어 모든 호출에서 재사용!

```
# 올바른 방법
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

Dataclass에서도 주의!

```
# 잘못된 방법 - 같은 함정!
@dataclass
class Student:
    name: str
    grades: List[int] = [] # 모든 학생이 같은 리스트 공유!

# 올바른 방법 - default_factory 사용
@dataclass
class Student:
    name: str
    grades: List[int] = field(default_factory=list)
```

문자열 Identity의 미스터리

```
>>> a = "hello"
>>> b = "hello"
>>> print(a is b)
True
```

```
>>> x = "hello world"
>>> y = "hello world"
>>> print(x is y)
False # 어?
```

```
>>> c = "hello!"
>>> d = "hello!"
>>> print(c is d)
False # 역시 False
```

abc 문자열 Identity - 더 이상한 현상

```
>>> # 한 줄에 쓰면?
>>> m, n = "hello!", "hello!"
>>> print(m is n)
True # 갑자기 True!

>>> # 세미콜론으로 구분하면?
>>> p = "hello!"; q = "hello!"
>>> print(p is q)
False # 다시 False
```

이유가 뭘까요? 🤔

문자열 인터닝 규칙

CPython의 문자열 인터닝(String Interning) 최적화:

1. 길이 0, 1인 모든 문자열은 자동 인터닝
2. 컴파일 타임에 결정
3. ASCII 문자, 숫자, 언더스코어만 포함된 문자열만 인터닝
4. 컴파일 단위가 중요:
 - `m, n = "hello!", "hello!"` : 하나의 statement → 같은 객체
 - 각각 다른 줄: 별도의 statement → 다른 객체

12 34 숫자 캐싱도 마찬가지

```
>>> a = 256
>>> b = 256
>>> print(a is b)
True

>>> c = 257
>>> d = 257
>>> print(c is d)
False # 각각 다른 줄에서는 False!

>>> # 한 줄에 세미콜론으로 쓰면?
>>> x = 257; y = 257
>>> print(x is y)
True # 같은 객체!
```

12 34 정수 캐싱 규칙

파이썬은 -5부터 256까지의 정수를 미리 생성해두고 재사용

```
>>> m = 100; n = 100
>>> print(m is n)
True
>>> p = 100
>>> q = 100
>>> print(p is q)
True # 범위 내 숫자는 항상 같음
```

257 이상: 코드 구성에 따라 다름 (컴파일 타임 최적화)

핵심 교훈

1. 직관에 의존하지 말고 검증하세요

- Wason Selection Task처럼 우리의 직관은 자주 틀립니다

2. Python의 내부 최적화를 이해하세요

- 문자열 인터닝, 정수 캐싱, 객체 참조

3. `is` vs `==` 의 차이를 명확히

- `is` : 객체 identity 비교
- `==` : 값 비교

4. 가변 객체 사용 시 항상 주의

- 기본 인수, 얇은 복사 문제

마무리

System 2 사고를 활성화하여:

- 코드의 예상치 못한 동작을 미리 예측
- 파이썬의 내부 동작 원리 이해
- 버그를 사전에 방지하는 습관 형성

기억하세요: 인간은 논리적이지 않지만,
좋은 습관으로 논리적 코딩이 가능합니다! 💪

참고 자료

- Python 조건식 평가 방식
- Python Truthy/Falsey 값
- WTF Python
- Daniel Kahneman - "Thinking, Fast and Slow"

감사합니다! 🙏

